arXiv:2505.04535v2 [cs.LG] 24 Jun 2025

FDA-OPT: Communication-Efficient Federated Fine-Tuning of Language Models

Michail Theologitis, Vasilis Samoladas, and Antonios Deligiannakis

***Abstract*—Federated Learning (FL) enables the utilization of vast, previously inaccessible data sources. At the same time, pre-trained Language Models (LMs) have taken the world by storm and for good reason. They exhibit remarkable emergent abilities and are readily adapted to downstream tasks. This opens one of the most exciting frontiers in FL: fine-tuning LMs. Yet, a persistent challenge in FL is the frequent, rigid communication of parameters—a problem magnified by the sheer size of these contemporary models. The FEDOPT family of algorithms has become the go-to approach for FL, relying on fixed but arbitrary intervals for model exchanges. Recently, the FDA algorithm prescribed a dynamic approach by monitoring the training progress. However, it introduced a hard-to-calibrate parameter and imposed a rigid synchronization scheme. In this work, we address these limitations by proposing the FDA-OPT family of algorithms—a unified generalization of both FDA and FEDOPT. Our experimental evaluation focuses on fine-tuning LMs on downstream NLP tasks and demonstrates that FDA-OPT outperforms FEDOPT even when it is configured with hyper-parameters specifically optimized for the latter. In other words, we show that FDA-OPT is a practical, drop-in replacement for FEDOPT in modern FL libraries and systems: it requires no additional configuration and delivers superior performance out of the box.**

***Index Terms*—Federated Learning, Deep Learning, Language Models.**

I. INTRODUCTION

IN today's data-driven landscape, leveraging data effectively has become both an opportunity and a challenge. Sensitive data, such as patient records in healthcare or personal information from mobile applications, is often siloed due to privacy concerns [1], [2] and stringent regulations [3]. While necessary for protecting user rights, these restrictions hinder the potential of machine learning solutions, as they prevent data from being centralized for training. Federated Learning (FL) [4], [5] emerges as a transformative paradigm by enabling models to be trained directly on the devices or organizations that own the data, bypassing restrictions and privacy concerns.

At the same time, pre-trained Language Models (LMs) have demonstrated extraordinary generalization power that enables them to be readily adapted—through techniques like fine-tuning—to a broad range of downstream tasks [6], [7]. By following the FL paradigm, these powerful models can be trained on vast, previously untapped data sources, enabling new breakthroughs and applications in natural language processing (NLP) [8], [9]. However, FL introduces a fundamental challenge: the frequent exchange of model updates. This issue is only exacerbated by the immense size of modern transformer-based models, making communication overhead a critical bottleneck [10].

To better understand this challenge, it is crucial to examine how FL operates in practice. In a typical FL setting, a central server orchestrates the training process across multiple data owners. Training proceeds in iterative rounds (Figure 1), each consisting of four key steps: broadcast, local training, collection, and aggregation. The server first broadcasts a global model to participating clients, who then train the model locally using their private data. After training, clients send their updates back to the server, who aggregates them into a new global model. These FL training rounds are repeated thousands of times until the model converges. Despite its conceptual simplicity, a critical open question remains: *When should the server collect model updates? When should a round end?*

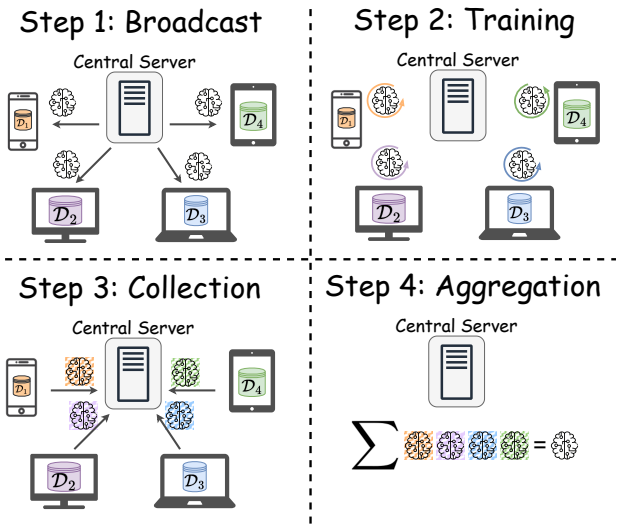


Fig. 1: A Federated Learning Round

The widely adopted answer to this question is the FEDOPT family of algorithms [11], which prescribe ending rounds after a fixed number of local steps or epochs. In this approach, each client processes its dataset one or more times before sending model updates back to the server. However, this method has two significant limitations. First, it assumes the training process is static, with configurations predefined throughout training, without taking into account real-time training dynamics. Second, the choice of the number of local steps is arbitrary and lacks justification. Lower values may improve convergence but skyrocket communication overhead, while higher values

The authors are with the School of Electrical and Computer Engineering, Technical University of Crete, Chania, 73100 Greece (e-mail: mtheologitis@tuc.gr; vsamoladas@tuc.gr; adeligiannakis@tuc.gr).

Manuscript received April 19, 2005; revised August 26, 2015.

reduce communication cost but risk unpredictable training behavior or, in many cases, non-convergence [12].

Recently, this trade-off has motivated the development of a dynamic approach to round termination: Federated Dynamic Averaging (FDA) [13]. Unlike static methods, FDA monitors the training state in real-time and adjusts the duration of each round accordingly. Specifically, FDA tracks the *model variance*—a gauge of whether training is progressing well or poorly—and ensures it remains below a prespecified threshold. As long as the variance is within this threshold, local training continues. When the variance exceeds the threshold, the round is terminated, and the server collects the models. This adaptive approach has been shown to significantly reduce communication overhead, achieving orders-of-magnitude improvements in communication compared to FEDOPT algorithms.

However, the original formulation of FDA [13] has certain limitations. First, it enforces a rigid synchronization scheme, requiring clients to exchange information with the server synchronously after every learning step, which can bottleneck a real-world FL system. Second, while FDA successfully eliminates the need to tune the number of local steps, it introduces a new parameter—the variance threshold—which is highly nontrivial to calibrate. In addition to these limitations, the experimental comparisons between FDA and FEDOPT were between best-case configurations for each algorithm, with different optimizers and hyper-parameters, raising the question whether FDA consistently outperforms FEDOPT in an apples-to-apples scenario.

In this work, we propose the FDA-OPT family of algorithms that generalizes both FDA and FEDOPT and addresses their core shortcomings. Most importantly, we demonstrate that our proposed FDA-OPT algorithms can be directly plugged into modern FL libraries as drop-in replacements for FEDOPT. They require no additional configuration and, in fact, perform better even when using hyper-parameters originally tuned for FEDOPT. Our contributions are as follows:

- We propose the FDA-OPT family of algorithms, a unified generalization of both FDA and FEDOPT. In doing so, we introduce a dynamic scheme for the variance threshold, removing the need for any manual configuration, and completely alleviate the original synchronization bottleneck.
- We show that FDA-OPT outperforms FEDOPT in communication-efficiency, achieving improvements of at least $2\times$ while operating under conditions optimized for FEDOPT (the competitor). Specifically, we manually identify the optimal hyper-parameter configurations for each FEDOPT algorithm, and then apply the same configurations to our proposed FDA-OPT counterparts. Remarkably, even with these "unfair" and "rigged" hyper-paremeters, FDA-OPT achieves $2\times$ greater communication-efficiency for the same model performance. This has the important implication that hyper-parameters proven effective for FEDOPT in the literature can be directly leveraged to configure our proposed algorithms.
- We show that all FDA-OPT algorithms converge to $5\times$–$10\times$ lower training loss than their FEDOPT counterparts within the same number of rounds.

The remainder of this paper is organized as follows. Section II introduces the foundational concepts of Federated Learning (FL), including the FEDOPT family of algorithms and the dynamic variance-based FDA approach. Section III presents our proposed FDA-OPT algorithm. Section IV outlines the experimental approach. Section V provides a detailed analysis of experimental results. Section VI reviews related work. Finally, Section VII summarizes the main findings of this work.

II. PRELIMINARIES

Before diving into our proposed algorithm, it is essential to understand the key concepts and methods that form its foundation. In this section, we break down the FL training process, explore the widely adopted FEDOPT family of algorithms, and introduce the recently proposed dynamic FDA approach. Together, these elements set the stage for understanding our subsequent contributions and the idea behind our generalized FDA-OPT algorithm in Section III.

A. Federated Learning Basics

Consider a scenario where we aim to train deep neural networks on data distributed across multiple devices or organizations—such as smartphones, hospitals, or sensors—that cannot be exchanged due to constraints like privacy or regulatory restrictions. Federated Learning (FL) addresses this challenge by enabling a set of clients, $\mathcal{K}$, to collaboratively train a deep learning model without sharing their data. Each client $k \in \mathcal{K}$ retains its private local dataset, $\mathcal{D}_k$.

Instead of centralizing data on a powerful cluster for training, FL reverses the traditional approach: training and computation are brought to the data, that is, to the clients. Each client trains a model locally and shares updates with a central server, which consolidates them into a common global model, $\boldsymbol{w} \in \mathbb{R}^d$. The collective goal is to minimize the overall training loss across all clients, which can be expressed as the following distributed optimization problem [10]:

$$\underset{\boldsymbol{w}\in\mathbb{R}^d}{\text{minimize}} \;\; F(\boldsymbol{w}) \triangleq \frac{1}{|\mathcal{K}|}\sum_{k\in\mathcal{K}} F_k(\boldsymbol{w}) \tag{1}$$

Here, $F_k(\boldsymbol{w}) \triangleq \mathbb{E}_{\zeta_k\sim\mathcal{D}_k}\left[\ell(\boldsymbol{w};\zeta_k)\right]$ is the local objective function for client k, and $\ell(\boldsymbol{w};\zeta_k)$ denotes the loss for a data sample ζ_k given the model $\boldsymbol{w}$.

B. Generalized Federated Averaging

The most widely used algorithm for solving (1) is Federated Averaging (FEDAVG) [5]. This method divides the FL training process into iterative rounds. At the beginning of round t, the server broadcasts the current global model $\boldsymbol{w}_t$ to a selected cohort of participating clients $\mathcal{S}_t \subseteq \mathcal{K}$. Each client $k \in \mathcal{S}_t$ initializes its local model as $\boldsymbol{w}_{t,0}^{(k)} = \boldsymbol{w}_t$, and performs τ local updates of Stochastic Gradient Descent (SGD) using its private dataset $\mathcal{D}_k$. After training, the client sends its updated model $\boldsymbol{w}_{t,\tau}^{(k)}$ back to the server. The server then aggregates

TABLE I: Overview of the FEDOPT and FDA-OPT families of algorithms. Depending on CLIENTOPT and SERVEROPT a different name is derived for the FL algorithm. FDA-OPT is our proposed algorithm, which will be introduced in Section III.

FEDOPT	FDA-OPT (ours)	CLIENTOPT	SERVEROPT
FEDAVG [5]	FDA-SGD	SGD	SGD (lr = 1.0)
FEDAVGM [15]	FDA-SGDM	SGD	SGDM [16]
FEDADAM [11]	FDA-ADAM	SGD	ADAM [14]
FEDADAMW [8]	FDA-ADAMW	SGD	ADAMW [17]
FEDADAGRAD [11]	FDA-ADAGRAD	SGD	ADAGRAD [18]

the updates—typically via averaging[1]—to form the new global model, $\boldsymbol{w}_{t+1}$.

An indirect yet highly effective way to mitigate the communication burden of FL is to accelerate convergence which led to efforts to incorporate accelerated optimization techniques—such as ADAM [14]—into the FL setting. However, FL clients typically lack enough data for robust statistical coverage and are often stateless, making adaptive optimization at the client level impractical. Instead, the key idea is to shift adaptive optimization to the server.

Specifically, clients still perform τ local update steps of SGD, but instead of sending back the final model weights themselves, each client computes its local *model change* (or *drift*)

$$\boldsymbol{\Delta}_{t,\tau}^{(k)} = \boldsymbol{w}_{t,\tau}^{(k)} - \boldsymbol{w}_{t,0}^{(k)},$$

and sends this to the server. The server then averages the updates across clients to obtain the average update direction

$$\boldsymbol{g}_{t,\tau} = -\frac{1}{|\mathcal{S}_t|} \sum_{k \in \mathcal{S}_t} \boldsymbol{\Delta}_{t,\tau}^{(k)},$$

which we refer to as the "pseudo-gradient". The term "pseudo" is used because even though $\boldsymbol{g}_{t,\tau}$ is not a traditional gradient—it was not derived from a loss function via backpropagation—it can still be treated in the same way [11]. The server then applies a server-side optimizer, SERVEROPT, to update the global model using this "pseudo"-gradient. Importantly, since $\boldsymbol{g}_{t,\tau}$ aggregates updates from multiple clients, it offers sufficient and trustworthy statistics, and the server can easily maintain the optimizer state across rounds. In effect, this addresses the two challenges that make adaptive methods impractical on clients: unreliable statistics, and statelessness.

This approach underpins the FEDOPT family of algorithms [11], which correspond to the non-highlighted lines of Algorithm 1 in Section III, alongside our proposed FDA-OPT algorithm (highlighted lines). We present them together, as FDA-OPT generalizes the core functionality of FEDOPT. Additionally, Table I summarizes the FEDOPT family. Notably, FEDAVG—which simply averages updates—is mathematically equivalent to SGD with a learning rate of 1.0.

[1]To be precise, the aggregation is a weighted average based on the number of samples $|\mathcal{D}_k|$ owned by each client; however, to simplify notation, and because the extension to a weighted average is straightforward, we will use averaging throughout this paper.

C. Federated Dynamic Averaging

Federated Dynamic Averaging (FDA) [13] is another algorithm for solving (1), with a key distinction from FEDAVG: instead of fixing the number of local steps τ per round, FDA dynamically decides when clients should stop training and return their models. Specifically, it monitors a metric called *model variance*, and terminates local training once this variance exceeds a predefined threshold Θ.

This paradigm change affects the transition between Step 2 and Step 3 in Figure 1—that is, when clients finish training and the server collects updates. Rather than enforcing a fixed number of SGD steps, FDA allows local training to continue as long as it is stable (i.e., variance remains low), and stops it early when instability is detected.

During each round t, clients periodically send small pieces of information to the server, which uses it to estimate the model variance and transmit this estimate back to the clients (see Figure 2). Specifically, after every local step i, each client $k \in \mathcal{S}_t$ computes its local model update $\boldsymbol{\Delta}_{t,i}^{(k)}$ and encodes it into a compact representation:

$$\mathbf{A}_{t,i}^{(k)} = \left[\|\boldsymbol{\Delta}_{t,i}^{(k)}\|_2^2,\ \mathrm{sk}(\boldsymbol{\Delta}_{t,i}^{(k)}) \right] \tag{2}$$

where $\mathrm{sk}(\cdot)$ is the AMS sketch operator [13], [19]. These updates are highly compressed (e.g., in our experiments, a model might be 1GB, but a sketch is just 10KB). The server averages these to form the global state $\mathbf{A}_{t,i}$, and computes an estimate of the variance $\nu_{t,i}$:

$$\nu_{t,i} = H(\mathbf{A}_{t,i}) \quad , \quad \mathbf{A}_{t,i} = \frac{1}{|\mathcal{S}_t|} \sum_{k \in \mathcal{S}_t} \mathbf{A}_{t,i}^{(k)}$$

Here, H is a function that maps the global state to a variance approximation (we refer to the original work for the full details [13]). Importantly, if a threshold violation is detected, $\nu_{t,i} > \Theta$, training stops, and the server collects models; otherwise, local training proceeds to the next SGD step, $i+1$, and Steps 2.1–2.4 repeat (Figure 2).

In many ways, FDA resembles the original formulation of FEDAVG. In the next section, we will extend its capabilities in a manner analogous to how FEDOPT generalized FEDAVG.

III. FDA-OPT: GENERALIZED FEDERATED DYNAMIC AVERAGING

In this section, we propose FDA-OPT, the unified generalization of both FDA and FEDOPT, with the following advancements:

1) **FL Adaptations**: We adapt the algorithm to accommodate FL-specific considerations, including client selection strategies and notation overlooked in the original design.
2) **Generalized Averaging**: We enhance server-side aggregation by incorporating adaptive and accelerated optimizers, extending beyond the original simple averaging scheme.
3) **Dynamic Variance Threshold**: We introduce a novel dynamic mechanism that automatically calibrates the variance threshold during training, eliminating the need for any manual tuning.
4) **Unified Configuration**: We ensure that FDA-OPT shares the exact same hyper-parameters as FEDOPT, allowing it

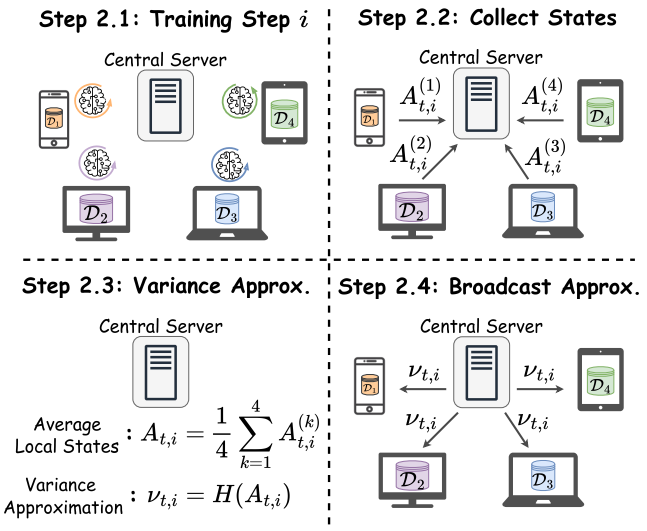


Fig. 2: Mechanics of local training, in round t, under FDA

to be configured using well-established settings from prior work.

5) **Alleviate Synchronization Bottleneck**: We relax the original rigid local-state synchronization requirement by allowing larger intervals between variance approximations.

A. Optimizers

In the original FDA algorithm, the server-side aggregation relied on a simple averaging scheme. However, extensive empirical evidence highlights the effectiveness of adaptive optimizers in FL [11]. Naturally, we extend the server-side aggregation step by introducing the use of arbitrary optimizers in FDA-OPT, denoted as SERVEROPT.

Intuitively, the client optimizer, CLIENTOPT, focuses on minimizing the local objective for each client based on its private data, while the server optimizer, SERVEROPT, operates from a global perspective. By employing adaptive or accelerated optimizers at the server, we can now take advantage of meaningful and accurate statistics, which are inherently unavailable at the client level.

B. Model Variance

In this subsection, we formalize the notion of model variance, provide intuition for its components, examine how it evolves during training, and explain how it can be interpreted and used as a signal of meaningful progress or instability.

Helpful Notation. Consider a training round at time t, involving a sampled subset of participating clients $\mathcal{S}_t = \{k_1, k_2, \ldots, k_N\}$, where $N = |\mathcal{S}_t|$. Moreover, let i be the local training step index. We can organize the client models into a matrix, $\boldsymbol{W}_{t,i} \in \mathbb{R}^{d \times N}$, defined as:

$$\boldsymbol{W}_{t,i} \triangleq \left[\boldsymbol{w}_{t,i}^{(k_1)}, \boldsymbol{w}_{t,i}^{(k_2)}, \ldots, \boldsymbol{w}_{t,i}^{(k_N)}\right]$$

This notation allows us to represent the collective state of the sampled clients in a compact and intuitive manner.

Additionally, as detailed in Section II-B, each client's model change, $\boldsymbol{\Delta}_{t,i}^{(k)}$, captures the drift after the i-th local training step. Furthermore, their average, $\boldsymbol{g}_{t,i}$, can be interpreted as a "pseudo"-gradient:

$$\boldsymbol{g}_{t,i} = -\frac{1}{|\mathcal{S}_t|}\sum_{k\in\mathcal{S}_t} \boldsymbol{\Delta}_{t,i}^{(k)} \quad , \quad \boldsymbol{\Delta}_{t,i}^{(k)} = \boldsymbol{w}_{t,i}^{(k)} - \boldsymbol{w}_{t,0}^{(k)}$$

Definition. The *model variance* quantifies the spread (or dispersion) of the client models around their average—indicating how compactly they are clustered in parameter space. At round t, after i local training steps, it can be written as [13]:

$$\mathrm{Var}(\boldsymbol{W}_{t,i}) = \overbrace{\frac{1}{|\mathcal{S}_t|}\sum_{k\in\mathcal{S}_t} \|\boldsymbol{\Delta}_{t,i}^{(k)}\|_2^2}^{1^{\text{st}}\text{ term}} - \underbrace{\|\boldsymbol{g}_{t,i}\|_2^2}_{2^{\text{nd}}\text{ term}} \tag{3}$$

Intuition. During a training round t, each client k updates its model, resulting in a local change $\boldsymbol{\Delta}_{t,i}^{(k)}$. The squared norm of this change, $||\boldsymbol{\Delta}_{t,i}^{(k)}||_2^2$, quantifies how far the client's model has moved from the start of the round (i.e., from the original global model). Essentially, this reflects the amount of information the client has learned locally. The first term in Equation (3), the average of these quantities across all participating clients, represents the *collective amount of information learned*.

However, a large first term does not necessarily mean that training is progressing well, as clients may be moving in conflicting directions, canceling out their updates by averaging. What ultimately matters is the global change $\boldsymbol{g}_{t,i}$ obtained after averaging—it foreshadows the information that will be retained in the global model. This is captured by the second term, $||\boldsymbol{g}_{t,i}||_2^2$, which inherently accounts for the *direction* of the local changes.

The variance, as defined in Equation (3), reflects the interplay between these two quantities serves as an insightful gauge of the state of the training progress:

1) **Low Variance** indicates either minimal local progress (both terms low), or substantial local progress in a cohesive and promising way (both terms high).
2) **High Variance** indicates significant local progress (high first term) but towards conflicting local minima (low second term).

Trends. What do we really mean by *low* or *high* variance? The answer is that these terms are inherently *relative* and task-specific, influenced by factors like the dataset, model size, optimizer, and hyper-parameters. To illustrate this, we investigate the value of the variance at the end of each FL round under the fixed-round termination schedule of FEDOPT. Figure 3 plots the variance at the exact moment when model updates are collected from the server. While the experimental setup—including the specific model and datasets—will be detailed later, we present this plot here to help readers intuitively grasp the variance as a metric and its trends.

This plot underscores the critical need to revisit the originally proposed static variance monitoring scheme. Variance trends are inherently dynamic, evolving throughout training, and thus require an adaptive approach to be captured effectively. Moreover, different algorithms exhibit vastly different

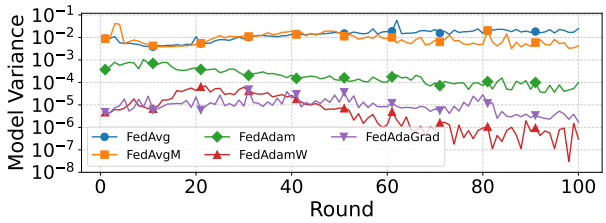


Fig. 3: Variance progression during the first 100 rounds of training with FEDOPT using RoBERTa on the MRPC dataset

variance magnitudes (please note that the y-axis is logarithmic) and behaviors. Hence, variance trends should only be interpreted within the context of the same training run. Additionally, the magnitude of the variance may evolve even for the same optimizer by 1-3 orders of magnitude, depending on whether the training is in its early, middle, or late stages.

All in all, *high* variance does not necessarily indicate that the model will converge to a suboptimal solution or that training is progressing poorly. Rather, high variance is interpreted as a sign of misalignment and instability when viewed *in relation* to earlier training rounds of the same run/optimizer.

C. The FDA-OPT Algorithm

High-Level Overview. Having established model variance as a key indicator of training dynamics—with distinct behaviors and trends—we now leverage these insights for a unique goal: *to extend local training duration beyond traditional limits without compromising convergence*. To this end, we the propose FDA-OPT, presented in Algorithm 1. The highlighted lines represent the core additions that distinguish FDA-OPT: namely, the logic for monitoring model variance and dynamically terminating rounds. These additions act as an adapter to the original FEDOPT framework (non-highlighted lines, also detailed in Section II-B), effectively generalizing it with an dynamic round termination scheme.

Operational Details. The algorithm begins with a user-provided local training duration τ. This value is expected to be based on well-established and empirically effective configurations from prior FEDOPT work. Then, FDA-OPT modifies this internally by extending it to a larger value $\tilde{\tau} \gg \tau$ (e.g., $\tilde{\tau} = 10 \cdot \tau$; see Line 2).

From this point onward, the algorithm proceeds using the standard round-based approach described in Section II-B. At the beginning of each round t, a cohort of clients $\mathcal{S}_t$ is sampled. These clients train locally for up to $\tilde{\tau}$ steps (Lines 3–9). At the end of training—whose actual length may vary—we compute the "pseudo"-gradient from model changes and update the global model accordingly (Lines 18 and 21). This process is repeated for a total of T rounds. What sets FDA-OPT apart is its variance monitoring logic.

Specifically, during local training, clients periodically synchronize compact state information (Lines 11–12) to approximate the model variance (Line 13). However, querying variance too frequently (e.g., after every local step) can completely bottleneck a real-world FL system. This is not due to communication volume (the local state states are small), but due to the synchronous nature of the operation. To mitigate this, we introduce a configurable set of step indices, $\mathcal{I}_{\text{query}} \subseteq [1, 2, ..., \tilde{\tau}]$, which determines the exact points where variance queries are performed (Line 10). In contrast to the original FDA formulation [13], which used $\mathcal{I}_{\text{query}} = [1, 2, ..., \tilde{\tau}]$ (i.e., checking after every step), our method leaves this set user-defined. In practice, we encourage it to be significantly sparser. As we will show later, in our setup the variance is queried only once per epoch.

Lastly, following the discussion on variance behavior in Section III-B, we allow the threshold value, Θ_t, to evolve across training rounds. Rather than using a fixed value, the threshold is updated dynamically based on the observed training dynamics. Specifically, we define a function THRESHOLDADJUST, which takes as input the current threshold Θ_t, the actual variance value at the end of the round, and the final step index s_t where the round terminated (Lines 19–20). Intuitively, these inputs capture the key information needed to answer two questions: (1) *was the current threshold too high or too low?* and (2) *did the round terminate at a reasonable point (e.g., $s_t = 1$ should raise concerns)?* In the next section, we will explore practical choices for this function. Notably, computing the variance in Line 19 requires no additional work, as it reuses quantities—the drifts and "pseudo"-gradient—that are already available (Lines 17–18).

D. Default Configuration for FDA-OPT

We adopt a conservative approach in the configurations below. These are the exact settings used in our experiments, as they yield stable and reliable performance across all tasks. While we believe there is room for improvement—potentially through more aggressive configurations (e.g., a larger linear coefficient of increase of τ)—we leave such tuning as a direction for future work.

Local Training Extension. We set $\tilde{\tau}$ to be twice the original τ plus a large constant. This constant is chosen to be eight times the size of a typical client dataset—denoted by e. Formally:

$$\tilde{\tau} = 2 \cdot \tau + 8 \cdot \lceil e \rceil, \quad \text{where} \quad e \approx \frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} |\mathcal{D}_k| \tag{4}$$

Variance Query Indices. Within each round, we query the variance once per epoch. This completely alleviates the synchronization bottleneck encountered in the original work [13]. We define:

$$\mathcal{I}_{\text{query}} = \{e, 2e, ..., \lfloor \tilde{\tau}/e \rfloor \cdot e\}$$

THRESHOLDADJUST. In general, variance within a round t tends to increase as local training progresses. We set the threshold Θ_t such that it is approximately equal to the expected variance around the midpoint of local training (i.e., at step $\lfloor \frac{\tilde{\tau}}{2} \rfloor$). Assuming linear growth, we can estimate the midpoint variance of the round $t+1$ using a simple proportion, based on known values from the current round t—specifically, the actual variance νar_t and termination step s_t computed in Lines 15 and 19 of Algorithm 1, respectively:

$$\nu ar_t \xrightarrow{\text{after}} s_t \quad \text{local steps}$$

$$\nu ar_{t+1}^{(est)} \xrightarrow{\text{after}} \frac{\tilde{\tau}}{2} \quad \text{local steps}$$

Algorithm 1 FDA-OPT

Input: Initial $\boldsymbol{w}_0$; CLIENTOPT, SERVEROPT; Total rounds T; Local training steps τ, specifically tuned for FEDOPT [11]

Set $\Theta_0 = -\infty$
Choose extended $\tilde{\tau} \gg \tau$ ▷ FDA-OPT extends local training
for each round $t = 0, \ldots, T-1$ **do**
  Sample a subset $\mathcal{S}_t \subseteq \mathcal{K}$ of clients
  Set $\boldsymbol{w}_{t,0}^{(k)} = \boldsymbol{w}_t$ for all $k \in \mathcal{S}_t$
  for each client $k \in \mathcal{S}_t$ **in parallel do**
    for each local step $i = 1, ..., \tilde{\tau}$ **do**
      Compute gradient estimate $\boldsymbol{g}_{t,i-1}^{(k)}$ of $\nabla F_k(\boldsymbol{w}_{t,i-1}^{(k)})$
      $\boldsymbol{w}_{t,i}^{(k)} = \text{CLIENTOPT}(\boldsymbol{w}_{t,i-1}^{(k)}, \boldsymbol{g}_{t,i-1}^{(k)})$
      If $i \in \mathcal{I}_{\text{query}}$ **then** ▷ Query variance now
        Construct local state $\mathbf{A}_{t,i}^{(k)}$ using (2)
        Aggregate states $\mathbf{A}_{t,i} = \frac{1}{|\mathcal{S}_t|} \sum_{k \in \mathcal{S}_t} \mathbf{A}_{t,i}^{(k)}$
        Variance approx. $\nu_{t,i} = H(\mathbf{A}_{t,i})$ ▷ See [13]
        If $\nu_{t,i} > \Theta_t$ **then** ▷ Threshold violation
          $s_t = i$ ▷ Round's final step index
          break ▷ Training round terminates
    $\boldsymbol{\Delta}_{t,s_t}^{(k)} = \boldsymbol{w}_{t,s_t}^{(k)} - \boldsymbol{w}_{t,0}^{(k)}$ ▷ Local model change
  $\boldsymbol{g}_{t,s_t} = -\frac{1}{|\mathcal{S}_t|} \sum_{k \in \mathcal{S}_t} \boldsymbol{\Delta}_{t,s_t}^{(k)}$ ▷ "Pseudo"-gradient
  $\nu ar_t = \text{Var}(\boldsymbol{W}_{t,s_t})$ ▷ Compute actual var. using (3)
  $\Theta_{t+1} = \text{THRESHOLDADJUST}(\nu ar_t, \Theta_t, s_t)$
  $\boldsymbol{w}_{t+1} = \text{SERVEROPT}(\boldsymbol{w}_t, \boldsymbol{g}_{t,s_t})$
return $\boldsymbol{w}_T$

With this linear prediction, we set the next round's threshold to the estimated variance at the midpoint of local training, $\nu ar_{t+1}^{(est)}$:

$$\Theta_{t+1} = \frac{\tilde{\tau}/2}{s_t} \cdot \nu ar_t$$

IV. EXPERIMENTAL APPROACH

In this section, we outline the methodology and rationale behind evaluating the proposed FDA-OPT family of algorithms against FEDOPT. Specifically, we detail the infrastructure, datasets, data partitioning strategies, and models used in our study.

A. Infrastructure

We conduct our experiments using the *transformers* library from Hugging Face [20] with PyTorch [21]. The code is available at GitHub[2]. All experiments are performed on a local cluster equipped with 2 NVIDIA A10 GPUs and 46 Intel(R) Xeon(R) Silver 4310 CPUs.

[2]https://github.com/miketheologitis/FDA-Opt

B. Tasks & Datasets

Natural Language Understanding (NLU) tasks are a core component of evaluating machine learning models in NLP. These tasks test a model's ability to understand, reason, and infer relationships between pieces of text. To conduct our experiments, we select six widely used datasets and tasks from the GLUE [22] benchmark: MRPC [23], SST-2 [24], RTE [22], QNLI [22], MNLI-m and MNLI-mm [25]—each corresponding to a distinct NLU task. Since the GLUE test sets are unpublished, we follow prior studies [26] and use the validation sets as the new test sets.

C. Data Partitioning

A critical aspect of FL is the distribution of data across clients, which is characterized by high heterogeneity. Most NLP datasets are designed for centralized machine learning settings. In this subsection, we describe how we partitioned these datasets for FL.

Clients. The scale of data federation is determined by the number of clients, which generally falls into two categories: *cross-silo* and *cross-device* [27]. In cross-silo FL, the number of clients is small—such as hospitals or financial institutions—often allowing all clients to participate in each training round. For this setting, we use the MRPC and RTE datasets. In contrast, cross-device FL involves a much larger pool of clients, such as mobile or IoT devices, where only a fraction of them can participate in each round. For this setting, we use the SST-2, QNLI, and MNLI datasets. Table II summarizes the client partitioning and sampling strategies across these datasets.

TABLE II: Client Partitioning Strategies per Dataset

	MRPC	RTE	SST-2	QNLI	MNLI-m	MNLI-mm
#Clients	10	10	100	250	1000	1000
Sample Size	10	5	10	10	10	10
Cross-Silo	✓	✓	✗	✗	✗	✗
Cross-Device	✗	✗	✓	✓	✓	✓

Label Distribution. One of the most common heterogeneity scenarios in FL is the presence of non-IID label distributions, where clients possess data with disproportionately distributed labels. For instance, one client may predominantly have samples of label A, while another may mostly contain label B. This phenomenon frequently occurs in real-world applications and is modeled using the Dirichlet distribution [28], [29], [30]. The degree of label imbalance is controlled by the concentration parameter α. Larger values of α result in more uniform distributions, with $\alpha \to \infty$ producing a perfectly uniform distribution. Conversely, as α approaches zero, the distributions become highly imbalanced, with each client predominantly holding samples from a single label. In our experiments, we set $\alpha = 1.0$, introducing a considerable level of heterogeneity.

Quantity Distribution. Another common heterogeneity scenario in FL is quantity-based imbalance, where clients possess vastly different amounts of data. We do not explicitly consider this scenario in our experiments. Nevertheless, while we aim to keep quantity distribution balanced, some low degree of

quantity imbalance is inevitably introduced due to the label-based non-IID scheme.

D. Models & Quality Baselines

In our experiments, we use the pre-trained transformer-based models RoBERTa [31] and DeBERTaV3 [32]. More specifically, we use the `roberta-base` and `microsoft/deberta-v3-base` checkpoints available through Hugging Face [20], which contain 125 million and 86 million parameters, respectively. Such encoder models are well-suited for FL, as they can be readily fine-tuned for a wide range of practical downstream NLP tasks [29] (e.g., sentiment analysis, textual entailment, etc.). We fine-tune both models with the FEDOPT and FDA-OPT algorithms on the datasets outlined in Section IV-B.

Of course, our ultimate goal in FL is to train accurate and robust models. Thus, as in most of the FL literature [11], we evaluate algorithms based on how efficiently they manage to do that. In other words, instead of comparing final accuracy scores or communication alone, we ask: *how many communication rounds does it take to train a model with a specific target quality?*

To define those targets in a consistent and task-agnostic way, we express them as percentages of the best-known centralized performance. Given both models' widespread adoption, their top-reported performance when fine-tuned in centralized settings is well-documented (see Table III). Naturally, due to the adverse conditions of data federation, FL performance is typically lower than centralized baselines. For example, the highest accuracy ever reported for RoBERTa when fine-tuned on MRPC is 90.2%; then, a target of 90% means attaining at least 81.18% accuracy ($90\% \cdot 90.2 = 81.18\%$).

TABLE III: Best-known accuracy scores (↑) for fine-tuning each model on each task in an ideal centralized setting. These values serve as target performance baselines

RoBERTa [31]				**DeBERTa** [32]	
MRPC↑	**RTE↑**	**SST-2↑**	**QNLI↑**	**MNLI-m↑**	**MNLI-mm↑**
90.2%	78.7%	94.8%	92.8%	90.6%	90.7%

E. Algorithms & Hyper-Parameters

We evaluate the most widely used algorithms from the FEDOPT family and compare their performance with our proposed FDA-OPT algorithms. To ensure a fair, apples-to-apples comparison, each FEDOPT algorithm is paired with its corresponding FDA-OPT variant using identical parameter configurations. For example, we compare FEDADAMW with FDA-ADAMW, keeping all optimizer settings fixed. The algorithmic pairs we compare are outlined in Table I.

Our goal is to conclusively demonstrate that FDA-OPT outperforms FEDOPT and that hyper-parameters proven effective for FEDOPT not only work for FDA-OPT but also yield better results. To show this, we first find the optimal hyper-parameter configuration for each FEDOPT algorithm[3]. Then, we apply these to our proposed FDA-OPT counterparts. Given this approach, evidence that FDA-OPT outperforms FEDOPT is particularly meaningful.

Lastly, unless otherwise specified, we set the number of local steps τ equal to one epoch—that is, $\tau = \lceil e \rceil$ from Equation (4)—as this is the most common and widely accepted choice in the FEDOPT literature [29], [11], [15]. Moreover, we use a batch size of 8 and train for an exhaustive number of rounds, $T \in \{100, \ldots, 1000\}$, ensuring that training continues well beyond likely convergence.

V. EXPERIMENTAL RESULTS & ANALYSIS

In this section, we evaluate the performance of FDA-OPT against FEDOPT across a variety of FL tasks, focusing on two key aspects: communication efficiency and convergence behavior, for the same model quality. Notably, our FDA-OPT algorithms operate under configurations optimized for their FEDOPT competitors—which makes the following observed improvements all the more noteworthy.

A. Main Findings

The main findings of our experimental analyses are the following:

- **Communication-Efficiency.** FDA-OPT demonstrates significant improvements in communication-efficiency. On average, it is $2.15\times$ more efficient in the cross-silo setting and $1.8\times$ in the cross-device setting to train the highest accuracy models.
- **Convergence.** FDA-OPT converges to 5–$10\times$ lower training loss than FEDOPT within the same number of rounds. The cross-silo setting is characterized by a sharp initial drop in loss, which is less pronounced in the cross-device setting.
- **Stability**. FDA-OPT is more robust across different initial local training step values, τ, while FEDOPT often fails to converge. Moreover, the communication improvements mirror those observed in the default case.

B. Communication-Efficiency

The communication overhead incurred by each algorithm is directly analogous to the number of rounds. For FEDOPT, communication between clients and the server occurs only at the end of each round. In FDA-OPT, there is additional communication due to the transmission of small sketches. However, the size of these sketches is negligible compared to the overall communication cost of transmitting language models (it is $\times 10^6$ smaller).

As is standard in the literature [33], we evaluate our algorithms by comparing their performance against predefined

[3]For each FEDOPT algorithm and dataset/task combination, we perform an exhaustive grid search to identify the best-performing hyper-parameters. For example, to optimize FEDADAM training RoBERTa on MRPC, we evaluate a 5×5 grid of client and server learning rate combinations—25 configurations in total—and select the one achieving the highest accuracy. Across all models, tasks, and optimizers, this process amounts to roughly 700 distinct training runs. While this substantial experimental effort is not emphasized in the main text due to space constraints, it underpins all reported results.

TABLE IV: Comparison of FEDOPT and FDA-OPT in communication-efficiency for training RoBERTa and DeBERTaV3 across different datasets to achieve target metrics. The table reports the number of rounds required to reach 85%, 90%, 95%, and 99% of the top-reported results in the literature (see Table III)—naturally, the fewer rounds, the better (↓). For example, in the bottom-left cell, FEDADAGRAD requires 29 rounds to achieve 90% of the best-reported accuracy for MRPC (i.e., at least $90\% \cdot 90.2 = 81.18\%$), while FDA-ADAGRAD achieves the same in 21 rounds, making it $1.4\times$ more efficient. Similarly, it has a speedup of $1.5\times$ for 95%. The "Avg. FDA-OPT Speedup" row is computed by averaging the individual FDA-OPT vs. FEDOPT speedup values across each column

	RoBERTa								**DeBERTaV3**			
	MRPC		**RTE**		**SST-2**		**QNLI**		**MNLI-m**		**MNLI-mm**	
	90%↓	**95%↓**	**90%↓**	**95%↓**	**95%↓**	**99%↓**	**90%↓**	**95%↓**	**85%↓**	**90%↓**	**85%↓**	**90%↓**
FEDAVG	26	30	67	103	8	93	35	71	12	16	12	15
FDA-SGD (ours)	**6**	**10**	**16**	**20**	**7**	**43**	**9**	**23**	**4**	**8**	**4**	**6**
FEDAVGM	21	31	111	182	30	91	38	81	29	35	29	35
FDA-SGDM (ours)	**5**	**16**	**41**	**126**	**12**	**41**	**11**	**44**	**12**	**15**	**12**	**15**
FEDADAM	20	40	118	**153**	12	42	11	22	11	12	11	12
FDA-ADAM (ours)	**7**	**11**	**72**	188	**8**	**25**	**6**	**20**	**9**	**10**	**9**	**10**
FEDADAMW	17	25	118	**160**	12	35	11	22	57	66	57	66
FDA-ADAMW (ours)	**14**	**16**	**71**	176	**8**	**14**	**6**	**20**	**43**	**54**	**42**	**52**
FEDADAGRAD	29	35	20	96	21	67	12	24	**8**	15	8	**11**
FDA-ADAGRAD (ours)	**21**	**23**	**18**	**54**	**7**	**24**	**6**	**20**	14	15	**6**	14
Avg. FDA-OPT Speedup	**2.8×**	**2.3×**	**2.3×**	**2×**	**1.9×**	**2.3×**	**2.6×**	**1.6×**	**1.7×**	**1.6×**	**1.9×**	**1.6×**

target metrics—such as accuracy (↑). The primary objective of the algorithms is to train a model that attains the target metric. Once they first do, we assess their communication efficiency by examining the number of FL rounds they required (↓). Table IV illustrates this evaluation.

Cross-Silo. The cross-silo setting includes RoBERTa trained on the MRPC and RTE datasets, as outlined in Table II.

- **RoBERTa on MRPC.** As per Table IV, FDA-OPT demonstrates superior communication-efficiency, with an average speedup of $2.3\times$-$2.8\times$ compared to FEDOPT. It is $2.8\times$ more efficient in reaching 90% of the target metric, and $2.3\times$ for 95%.
- **RoBERTa on RTE.** Similar trends are observed for the RTE dataset. FDA-OPT achieves average speedups of $2\times$-$2.3\times$ over FEDOPT, with specific gains of $2.3\times$ for 90%, and $2\times$ for 95% of the target metrics.

When the goal is to train the most accurate model—defined here as achieving 95% of the top-reported metrics—FDA-OPT is, on average, $2.15\times$ more communication-efficient in the cross-silo setting.

Cross-Device. In the cross-device setting, we analyze performance of training RoBERTa on the SST-2 and QNLI datasets, and DeBERTaV3 on MNLI-m and MNLI-mm, as outlined in Table II.

- **RoBERTa on SST-2.** FDA-OPT consistently outperforms FEDOPT, achieving an improvement in communication-efficiency of $1.9\times$-$2.3\times$. Specifically, FDA-OPT is $1.9\times$ more efficient in achieving 90% of the target metric, and $2.3\times$ for 99%.
- **RoBERTa on QNLI.** Again, FDA-OPT exhibits speedups ranging from $1.6\times$-$2.6\times$. It is $2.6\times$ more efficient at 90%, and $1.6\times$ at 95% of the target metric.
- **DeBERTaV3 on MNLI-m.** Similarly, FDA-OPT has improvements of $1.7\times$ and $1.6\times$, corresponding to 85% and 90% of the target metrics, respectively.
- **DeBERTaV3 on MNLI-mm.** Lastly, FDA-OPT is $1.7\times$ and $1.6\times$ more communication-efficient than FEDOPT, for attaining 85% and 90% of the target accuracies, respectively.

Similarly, in the cross-device setting, FDA-OPT achieves an average improvement in communication-efficiency of $1.8\times$ when training models to attain the highest possible accuracy targets.

C. Convergence

Analyzing training loss progression is key to understanding the convergence behavior of FL algorithms. It provides insights into how quickly and effectively models learn—essentially, their ability to minimize objectives. In this subsection we analyze Figure 4 which compares the performance of FDA-OPT and FEDOPT on two representative datasets from cross-silo and cross-device settings. The conclusions drawn regarding convergence pace and quality from these two datasets are indicative of the broader trends observed across all experiments.

Cross-Silo. On the MRPC dataset—which simulates a cross-silo setting following Table II—all FDA-OPT algorithms demonstrate significantly faster convergence than their FEDOPT counterparts (Figure 4). For instance, FDA-SGD achieves a training loss $100\times$ smaller than FEDAVG within the first 100 rounds. Additionally, both FDA-SGDM and FDA-ADAMW exhibit sharp drops within the initial 30–40 rounds, outperforming FEDAVGM and FEDADAMW, respectively, with FDA-SGDM stabilizing at a loss $10\times$ smaller than FEDAVGM. Meanwhile, FDA-ADAM converges to a loss $5\times$ smaller than FEDADAM, and FDA-ADAGRAD consistently maintains a $5\times$ lower loss than FEDADAGRAD after the initial rounds.

Cross-Device. On the SST-2 dataset—representing a cross-device setting as outlined in Table II—all FDA-OPT algorithms consistently achieve 5–$10\times$ lower training loss compared to their FEDOPT counterparts (Figure 4). Unlike the cross-silo case, the initial drop in training loss is less pronounced, which is expected since each round involves only a

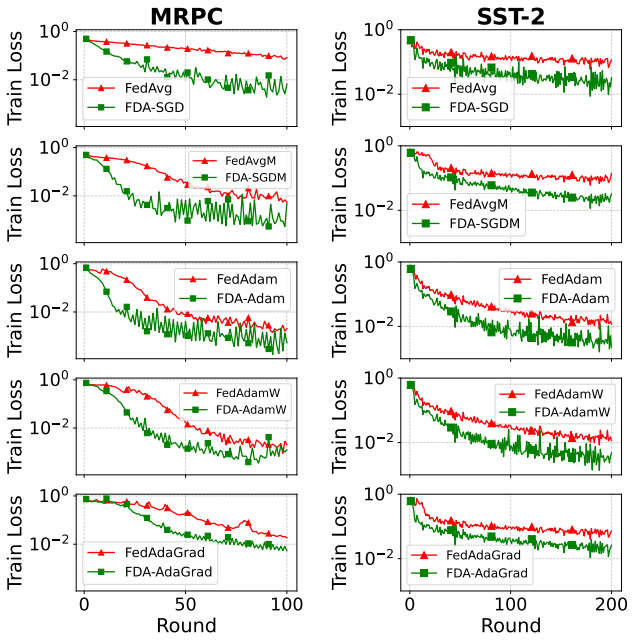


Fig. 4: Training Loss progression of training RoBERTa with FDA-OPT vs. FEDOPT on MRPC (left) and SST-2 (right)

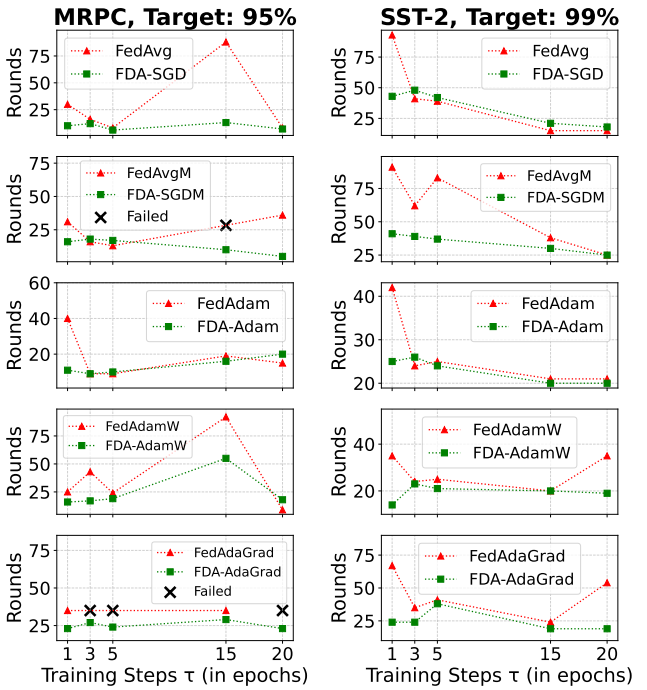


Fig. 5: Training RoBERTa with FDA-OPT vs. FEDOPT for varying initial local training steps τ (measured in epochs). We report the number of rounds required to attain the target accuracy. Failure to converge is marked with "×"

small subset of clients. This leads to initial updates that are less representative of the overall training distribution.

D. Stability to Local Training Extension

Empirical evidence suggests that increasing the number of local training steps τ can lead to degraded performance or non-convergence altogether [33], [12]. Since our scheme extends local training even further ($\tilde{\tau} \gg \tau$), it is crucial to examine whether this extension introduces any instability. To this end, we train RoBERTa on MRPC (cross-silo) and SST-2 (cross-device) using a range of values for τ.

The results, shown in Figure 5, reveal two key insights. First, in every one of the 10 subplots, FDA-OPT consistently requires fewer training rounds than FEDOPT. Across all 50 individual setups, FDA-OPT outperforms FEDOPT in 40 cases—often by a large margin. Conversely, FEDOPT shows only marginal improvements in the remaining 10 cases. Second, and most importantly, FDA-OPT converges reliably across all experiments. In contrast, FEDOPT fails to converge in 4 out of 50 (specifically, FEDAVGM and FEDADAGRAD on MRPC). This is particularly problematic in FL, where non-convergence is almost impossible to detect and may lead to severe, and exploding communication cost.

These findings highlight that FDA-OPT is not only more efficient, but also inherently stable and *safe* for extended local training intervals—a direct result of monitoring the variance.

VI. RELATED WORK

This section reviews related work on communication-efficient FL.

Round Termination. Several works have explored strategies for determining the round termination intervals in FL. The study in [34] aims to find a fixed, optimal interval by balancing local computation and aggregation constraints. In contrast, [35] analyzes the trade-off between fast convergence and round completion rates under strongly convex client objectives, leading to a decaying round duration scheme. Similarly, [36] minimizes convergence error with respect to wall-clock time by progressively decreasing the round duration. On the other hand, [37] focuses on minimizing communication rounds for a fixed number of model updates, which results in an increasing sequence of round duration intervals. Unlike these approaches, which rely on predefined schedules—whether fixed, decaying, or increasing—FDA-OPT makes round termination decisions dynamically, in real-time, based on the state of the training.

Convergence. The most direct way to alleviate the communication burden in FL is to accelerate convergence. This motivated the development of the FEDOPT family (Section II-B). However, FL algorithms still struggle with high heterogeneity, where different clients converge to disparate and often conflicting local minima [38], [11], [39], ultimately slowing down convergence. To address this, SCAFFOLD introduced control variates to correct client drift [38]. Similarly, the Mime framework [40] leverages both control variates and server statistics. FEDPROX[41] tackles heterogeneity by adding an L^2 regularization term. Notably, it has been demonstrated that FEDOPT outperforms FEDPROX in NLP tasks similar to ours [29]. Lastly, FEDDYN [42] introduces a dynamic regularizer, ensuring that client convergence aligns with the stationary point of the global objective. Importantly, FDA-OPT is orthogonal to these optimization methods as we focus on a complementary aspect: when to terminate rounds.

PEFT. Parameter-efficient fine-tuning (PEFT) [43], [44], [45], [46] trains only a small subset of a model's parameters while keeping the rest frozen. One of the most widely used approaches, LoRA [45], injects low-rank adaptation matrices into a transformer's frozen attention layers, reducing the number of trainable parameters to $< 1\%$. Given the communication constraints in FL, recent studies have explored the applicability of PEFT methods in this setting [26], [47]. However, techniques like LoRA struggle with the high heterogeneity present in FL [47]. SLoRA [48] addresses this issue by leveraging sparse fine-tuning [49] to initialize the injected parameters more effectively. These PEFT methods can be readily integrated with FDA-OPT, as the choice of trainable parameters, $\mathbf{w}$, remains user-defined.

Compression. Another way to reduce communication in FL is to compress transmitted information using techniques like quantization [50] and sparsification [51], [52]. A comprehensive survey of these methods is provided in [53]. Notably, compression is orthogonal to our approach and can be seamlessly integrated with FDA-OPT.

VII. CONCLUSION

In this work, we introduced the FDA-OPT family of algorithms, a unified generalization of both FDA and FEDOPT, addressing their core limitations. We empirically prove that FDA-OPT is more communication efficient and reliable than FEDOPT. Furthermore, through carefully designed experiments, we demonstrating that FDA-OPT can seamlessly replace FEDOPT, as it can be directly configured using settings from the FEDOPT literature. To this end, each comparison between the two algorithmic families was conducted using the best-performing configurations for our competitor, FEDOPT—showing that well-established settings from the literature can be applied to our algorithms without modification. Importantly, even under these "unfair" conditions, FDA-OPT achieves at least $2\times$ greater communication-efficiency on average and consistently converges to $5\times$–$10\times$ lower training loss. These findings hint at significant practical implications for improving modern FL libraries and real-world deployments.

A. Hyper-parameter Selection

In this Appendix, we detail the approach for identifying the optimal configuration for each FEDOPT algorithm when training a specific model on a specific dataset. Algorithms in the FEDOPT family are highly sensitive to both client and server learning rates, and their interaction is often task-specific and unpredictable. We conduct a grid search to determine the best hyper-parameters for each dataset and algorithm. The final selections based on this process are summarized in Table V.

As a representative example, consider the RoBERTa model trained MRPC dataset. We tune learning rates using a commonly adopted search grid [33] (Figure 6). Specifically we investigate server and client learning rates in $\{10^{-5}, 10^{-4}, \ldots, 10^{-1}\}$ for FEDADAGRAD, following well-established guidelines [18]. We then train RoBERTa with FEDADAGRAD for $T = 100$ rounds, recording the highest accuracy achieved across all rounds. As shown in Figure 6, the best-performing configuration for FEDADAGRAD on MRPC reached 89% accuracy with server and client learning rates of 10^{-4} and 10^{-5}, respectively. These values are taken as optimal for training RoBERTa with FEDADAGRAD on MRPC.

We then apply these optimal learning rates to the FDA-OPT counterpart. Specifically, when training RoBERTa with FDA-ADAGRAD on MRPC, we use the server and client learning rates of 10^{-4} and 10^{-5}, respectively, as determined from the above analysis. While these values are most likely not optimal for FDA-ADAGRAD, our goal is to conclusively demonstrate that FDA-OPT outperforms FEDOPT and that hyper-parameters proven effective for FEDOPT not only work for FDA-OPT but also yield better results. This has the powerful implication that we can seamlessly replace all FDA-OPT algorithms with FEDOPT.

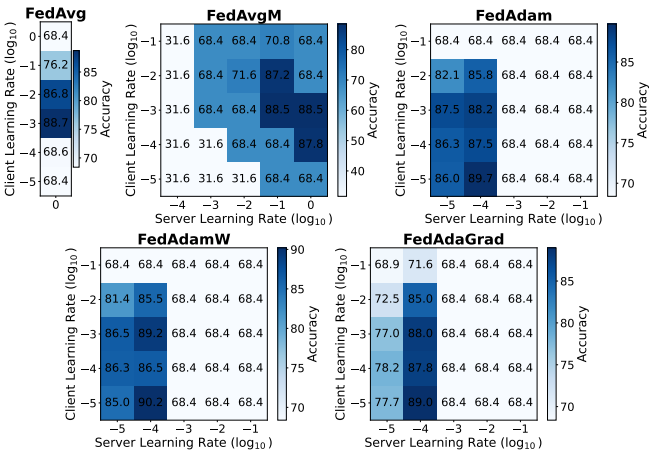


Fig. 6: The highest testing accuracy of RoBERTa on the MRPC dataset during the first 100 rounds of training, across various client and server optimizer learning rates. The MRPC test set is imbalanced, with 68.4% of the pairs labeled as positive and 31.6% as negative. As a result, these two accuracy values indicate that the model is always choosing one class, and should be treated as signs of non-convergence. Notably, FEDAVG has an averaging aggregation scheme

TABLE V: Best-performing learning rates ($\log_{10}$) for training RoBERTa and DeBERTaV3 on different datasets using each FEDOPT algorithm. SERVEROPT is shown on top and CLIENTOPT on the bottom within each cell. For example, the bottom-left cell corresponds to training RoBERTa on MRPC with FEDADAGRAD, where the optimal learning rates are 10^{-4} for server-side ADAGRAD and 10^{-5} for client-side SGD

	RoBERTa				DeBERTaV3
	MRPC	**RTE**	**SST-2**	**QNLI**	**MNLI-m/mm**
FEDAVG	0 −3	0 −3	0 −3	0 −3	0 −2
FEDAVGM	0 −3	−1 −3	0 −4	0 −3	0 −3
FEDADAM	−4 −4	−5 −5	−4 −4	−4 −3	−4 −2
FEDADAMW	−4 −5	−5 −5	−4 −4	−4 −3	−4 −4
FEDADAGRAD	−4 −5	−4 −3	−4 −4	−4 −3	−4 −2

REFERENCES

[1] G. Yan, H. Wang, X. Yuan, and J. Li, "Fedrola: Robust federated learning against model poisoning via layer-based aggregation," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3667–3678.

[2] Y. Jiang, X. Luo, Y. Wu, X. Zhu, X. Xiao, and B. C. Ooi, "On data distribution leakage in cross-silo federated learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3312–3328, 2024.

[3] D. Chai, L. Wang, L. Yang, J. Zhang, K. Chen, and Q. Yang, "A survey for federated learning evaluations: Goals and measures," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 10, pp. 5007–5024, 2024.

[4] Q. Li, Z. Wen, Z. Wu, S. Hu, N. Wang, Y. Li, X. Liu, and B. He, "A survey on federated learning systems: Vision, hype and reality for data privacy and protection," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 4, pp. 3347–3366, 2023.

[5] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, A. Singh and J. Zhu, Eds., vol. 54. PMLR, 20–22 Apr 2017, pp. 1273–1282.

[6] F. Xu, Q. Lin, J. Han, T. Zhao, J. Liu, and E. Cambria, "Are large language models really good logical reasoners? a comprehensive evaluation and beyond," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 4, pp. 1620–1634, 2025.

[7] L. Hu, Z. Liu, Z. Zhao, L. Hou, L. Nie, and J. Li, "A survey of knowledge enhanced pre-trained language models," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 4, pp. 1413–1430, 2024.

[8] H. Woisetschläger, A. Erben, S. Wang, R. Mayer, and H.-A. Jacobsen, "Federated fine-tuning of llms on the very edge: The good, the bad, the ugly," in *Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning*, ser. DEEM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 39–50.

[9] F. Wu, Z. Li, Y. Li, B. Ding, and J. Gao, "Fedbiot: Llm local fine-tuning in federated learning without full model," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3345–3355.

[10] H. Woisetschläger, A. Erben, S. Wang, R. Mayer, and H.-A. Jacobsen, "A survey on efficient federated learning methods for foundation model training," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, K. Larson, Ed. International Joint Conferences on Artificial Intelligence Organization, 8 2024, pp. 8317–8325, survey Track.

[11] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.

[12] H. Yu, S. Yang, and S. Zhu, "Parallel restarted sgd with faster convergence and less communication: demystifying why model averaging works for deep learning," in *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI'19/IAAI'19/EAAI'19. AAAI Press, 2019.

[13] M. Theologitis, G. Frangias, G. Anestis, V. Samoladas, and A. Deligiannakis, "Communication-efficient distributed deep learning via federated dynamic averaging," in *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25 - March 28*. OpenProceedings.org, 2025, p. 411–424.

[14] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[15] T. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *CoRR*, 2019.

[16] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," in *Proceedings of the 30th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, S. Dasgupta and D. McAllester, Eds., vol. 28, no. 3. Atlanta, Georgia, USA: PMLR, 17–19 Jun 2013, pp. 1139–1147.

[17] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[18] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of Machine Learning Research*, vol. 12, no. 61, pp. 2121–2159, 2011.

[19] G. Cormode and M. Garofalakis, "Sketching streams through the net: Distributed approximate query tracking," in *Proceedings of the 31st International Conference on Very Large Data Bases*, ser. VLDB '05. VLDB Endowment, 2005, p. 13–24.

[20] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Q. Liu and D. Schlangen, Eds. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45.

[21] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[22] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, T. Linzen, G. Chrupała, and A. Alishahi, Eds. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 353–355.

[23] W. B. Dolan and C. Brockett, "Automatically constructing a corpus of sentential paraphrases," in *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*, 2005.

[24] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, D. Yarowsky, T. Baldwin, A. Korhonen, K. Livescu, and S. Bethard, Eds. Seattle, Washington, USA: Association for Computational Linguistics, Oct. 2013, pp. 1631–1642.

[25] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2018, pp. 1112–1122. [Online]. Available: http://aclweb.org/anthology/N18-1101

[26] Z. Zhang, Y. Yang, Y. Dai, Q. Wang, Y. Yu, L. Qu, and Z. Xu, "FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models," in *Findings of the Association for Computational Linguistics: ACL 2023*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 9963–9977.

[27] Y. Jiang, X. Luo, Y. Wu, X. Zhu, X. Xiao, and B. C. Ooi, "On data distribution leakage in cross-silo federated learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3312–3328, 2024.

[28] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-iid data silos: An experimental study," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 2022, pp. 965–978.

[29] B. Y. Lin, C. He, Z. Ze, H. Wang, Y. Hua, C. Dupuy, R. Gupta, M. Soltanolkotabi, X. Ren, and S. Avestimehr, "FedNLP: Benchmarking federated learning methods for natural language processing tasks," in *Findings of the Association for Computational Linguistics: NAACL 2022*, M. Carpuat, M.-C. de Marneffe, and I. V. Meza Ruiz, Eds. Seattle, United States: Association for Computational Linguistics, Jul. 2022, pp. 157–175.

[30] Z. Qin, S. Deng, M. Zhao, and X. Yan, "Fedapen: Personalized cross-silo federated learning with adaptability to statistical heterogeneity," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1954–1964.

[31] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized BERT pretraining approach," *CoRR*, vol. abs/1907.11692, 2019.

[32] P. He, J. Gao, and W. Chen, "Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing," 2023.

[33] J. Wang, Z. Charles, Z. Xu, G. Joshi, H. B. McMahan, B. A. y Arcas, M. Al-Shedivat, G. Andrew, S. Avestimehr, K. Daly, D. Data, S. N. Diggavi, H. Eichner, A. Gadhikar, Z. Garrett, A. M. Girgis, F. Hanzely, A. Hard, C. He, S. Horváth, Z. Huo, A. Ingerman, M. Jaggi, T. Javidi, P. Kairouz, S. Kale, S. P. Karimireddy, J. Konečný, S. Koyejo, T. Li, L. Liu, M. Mohri, H. Qi, S. J. Reddi, P. Richtárik, K. Singhal, V. Smith, M. Soltanolkotabi, W. Song, A. T. Suresh, S. U. Stich, A. Talwalkar, H. Wang, B. E. Woodworth, S. Wu, F. X. Yu, H. Yuan, M. Zaheer, M. Zhang, T. Zhang, C. Zheng, C. Zhu, and W. Zhu, "A field guide to federated optimization," *CoRR*, vol. abs/2107.06917, 2021.

[34] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, and K. Chan, "Adaptive federated learning in resource constrained edge computing systems," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1205–1221, 2019.

[35] J. Mills, J. Hu, and G. Min, "Faster federated learning with decaying number of local sgd steps," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 7, p. 2198–2207, jul 2023.

[36] J. Wang and G. Joshi, "Adaptive communication strategies to achieve the best error-runtime trade-off in local-update sgd," *Systems and Machine Learning (SysML) Conference*, 2018.

[37] F. Haddadpour, M. M. Kamani, M. Mahdavi, and V. R. Cadambe, "Local sgd with periodic averaging: Tighter analysis and adaptive synchronization," in *Neural Information Processing Systems*, 2019.

[38] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, H. D. III and A. Singh, Eds., vol. 119. PMLR, 13–18 Jul 2020, pp. 5132–5143.

[39] P. Wenig and T. Papenbrock, "Datagossip: A data exchange extension for distributed machine learning algorithms," in *Proceedings of the 25th International Conference on Extending Database Technology, EDBT 2022, Edinburgh, UK, March 29 - April 1, 2022*, J. Stoyanovich, J. Teubner, P. Guagliardo, M. Nikolic, A. Pieris, J. Mühlig, F. Özcan, S. Schelter, H. V. Jagadish, and M. Z. 0001, Eds., 2022.

[40] S. P. Karimireddy, M. Jaggi, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "Mime: Mimicking centralized stochastic algorithms in federated learning," 2021.

[41] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of the Third Conference on Machine Learning and Systems, MLSys 2020, Austin, TX, USA, March 2-4, 2020*, I. S. Dhillon, D. S. Papailiopoulos, and V. Sze, Eds. mlsys.org, 2020.

[42] D. A. E. Acar, Y. Zhao, R. Matas, M. Mattina, P. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *International Conference on Learning Representations*, 2021.

[43] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds. Online: Association for Computational Linguistics, Aug. 2021, pp. 4582–4597.

[44] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 3045–3059.

[45] E. J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[46] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 1–9.

[47] Y. Sun, Z. Li, Y. Li, and B. Ding, "Improving loRA in privacy-preserving federated learning," in *The Twelfth International Conference on Learning Representations*, 2024.

[48] S. Babakniya, A. Elkordy, Y. Ezzeldin, Q. Liu, K.-B. Song, M. EL-Khamy, and S. Avestimehr, "SLoRA: Federated parameter efficient fine-tuning of language models," in *International Workshop on Federated Learning in the Age of Foundation Models in Conjunction with NeurIPS 2023*, 2023.

[49] A. Ansell, E. Ponti, A. Korhonen, and I. Vulić, "Composable sparse fine-tuning for cross-lingual transfer," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 1778–1796.

[50] N. Shlezinger, M. Chen, Y. Eldar, H. V. Poor, and S. Cui, "Uveqfed: Universal vector quantization for federated learning," *IEEE Transactions on Signal Processing*, vol. 69, pp. 500–514, 01 2021.

[51] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, M. Palmer, R. Hwa, and S. Riedel, Eds. Copenhagen, Denmark: Association for Computational Linguistics, Sep. 2017, pp. 440–445.

[52] D. Basu, D. Data, C. Karakus, and S. Diggavi, *Qsparse-local-SGD: distributed SGD with quantization, sparsification, and local computations*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[53] Z. Wang, M. Wen, Y. Xu, Y. Zhou, J. H. Wang, and L. Zhang, "Communication compression techniques in distributed deep learning: A survey," *Journal of Systems Architecture*, vol. 142, p. 102927, 2023.